

FreshMart Sales Analytics

Challenges & Corrections Log

This document summarizes the real technical and analytical issues I ran into across both the data cleaning (Phase 1) and EDA (Phase 2) stages of this project, and how each was resolved.

Phase 1 — Data Cleaning

Challenge	Correction
<code>`pd.to_datetime(format="mixed")`</code> silently misread day-first dates as month-first (e.g. <code>`"04-06-1997"`</code> read as April 6 instead of June 4), since it defaults to <code>`dayfirst=False`</code> . It never errored, so the wrong dates would have gone unnoticed.	Wrote a custom parser that matches each date string to its exact format pattern by separator/structure before parsing, instead of letting pandas guess. Applied it to every date column in every table.
Casting <code>`customer_id`</code> to text with <code>`.astype(str)`</code> silently turned missing values into the literal string <code>`"nan"`</code> , which would have made every anonymous transaction look like a known customer.	Used <code>`.astype("Int64").astype("string")`</code> instead, which preserves true missing values through the type conversion. Verified the null count matched before and after.
<code>`product_id`</code> was left as <code>`int64`</code> in the Products table but cast to <code>`string`</code> in Transactions and Returns — a silent dtype mismatch that would make any merge between them return zero or partial matches.	Standardized <code>`product_id`</code> to <code>`string`</code> across all three tables.

Phase 2 — Exploratory Data Analysis

Challenge	Correction
CSV files don't retain dtype metadata — every fresh <code>`pd.read_csv()`</code> in a new notebook silently reverted my cleaned dtypes back to raw types (dates back to text, nullable IDs back to plain floats).	Added a short "restore dtypes" step at the top of the EDA notebook, re-applying the same date parser and type casts before any analysis.
Checking whether larger stores sell more by correlating <code>`total_sqft`</code> against individual transaction <code>`quantity`</code> gave a near-zero correlation (-0.001) — because store size is a store-level property, not a per-transaction one, so this measured basket size, not store performance.	Aggregated to one row per store first (total quantity per store), then correlated against store size. The true relationship was a strong 0.97, the opposite conclusion.
Merging the Returns table directly with the Transactions table on <code>`product_id`</code> (with no other matching key) caused a cartesian explosion — 6,881 return rows fanned out into over 11 million rows, since every return matched every transaction of the same product. Return rates computed from this came out above 100%, which is impossible.	Aggregated total sold and total returned quantity separately by product first, then merged the two small summaries. Return rates came back sane (max ~14%).
Some <code>`sales_region`</code> values were only partially cleaned in Phase 1 — casing was fixed (<code>`SOUTHEAST`</code> → <code>`Southeast`</code>) but different spacing variants (<code>`Mid West`</code> vs <code>`Midwest`</code>) weren't merged, silently splitting one true region into multiple smaller bars on the region chart.	Wrote a normalization function that merges spacing/casing variants into one canonical region name, and re-ran the region-level analysis.
A handful of transactions (~0.1%) referenced a <code>`store_id`</code> that didn't exist in the Stores table — an invalid foreign key that would inflate any transaction count not explicitly merged against Stores.	Filtered <code>`tr_clean`</code> to only valid, known store IDs and documented the row count removed.
One store's <code>`grocery_sqft`</code> was filled with a Phase 1 median-imputed value that was physically impossible — larger than the store's own <code>`total_sqft`</code> .	Identified and excluded the store from the size-vs-volume correlation specifically, with a note explaining why.

Review process

I did all the cleaning decisions, EDA, and code myself. Throughout the project, I also used Claude as a second reviewer — going through each notebook the way a second set of eyes would in a code review, specifically to catch anything that could quietly produce a wrong or misleading result. Every issue listed above was either caught during that review or caught by me and independently verified with Claude before I applied the fix. In each case, I confirmed the fix worked by re-checking row counts, null counts, or the resulting numbers before moving on.

Closing note

Most of these issues never raised an error — they ran cleanly and produced a number that looked plausible. That was the actual lesson of this project: a wrong result rarely announces itself, so checking why a number makes sense mattered more than just getting code to run without errors.